

JasOS

0.02 second, 512 kilobytes

เนื้อเรื่อง

แจสโม่หัวหน้า JAS LAB กำลังพัฒนาระบบปฏิบัติการ JasOS เพื่อใช้งานในระบบคอมพิวเตอร์ ซึ่งได้ขอร้องให้คุณ และจัสมินมาช่วยทำในส่วนของระบบการจัดการหน่วยความจำ

JasOS ใช้เทคนิคการทำงานที่มีชื่อว่า Virtual Memory และ Paging สำหรับการจัดสรรหน่วยความจำให้กับโปรเซสต่าง ๆ เป็นเทคนิคที่ทำให้ระบบสามารถเสมือนใช้งานหน่วยความจำได้เกินกว่าความเป็นจริง ในการจัดสรรหน่วยความจำของระบบจะแบ่งส่วนของหน่วยความจำแบบเป็น Page (การแบ่งเชิงตรรกะ) และ Frame (หน่วยความจำจริง) เพื่อป้องกันการเข้าถึงหน่วยความจำข้ามโปรเซส CPU จะสร้างหมายเลขที่อยู่เชิงตรรกะ (Logical address) เพื่อเข้าถึงแผนการใช้ หมายเลขที่อยู่เชิงกายภาพ (Physical address) โดยมีองค์ประกอบดังนี้

$$\text{logical address} = \text{page number} \cdot 256 + \text{offset}$$

โดยที่ page number ($0 \leq \text{page number} \leq 4096$) และแต่ละ page มีความกว้างอยู่ที่ 256 ช่อง

Page Table เป็นตาราง Hash table ขนาด 17 ช่องที่ใช้เพื่อเก็บสถานะของ page ที่กำลังใช้งาน หรือที่ถูกดึงออกไปแล้วโดย โดยประกอบด้วย page number, frame number, dirty (บอกว่า page นี้ถูกเขียนระหว่างใช้งานหรือไม่), access counter (จำนวนการเข้าถึงหน่วยความจำนี้), cached value (ค่าที่ถูกเขียน) และสถานะของ page โดยสถานะของ page ที่เก็บประกอบด้วย Empty (ไม่เคยใช้มาก่อน), Occupied (กำลังใช้งาน), Deleted (ลบไปแล้ว) โดย Page table ใช้ Hash function และเทคนิคการป้องกันการชนกันของ page ในสถานการณ์ที่แทรก page ใหม่ดังนี้

สำหรับการชนกันของ page ให้หาค่า index ดังนี้ สำหรับรอบที่ $i = 0, 1, 2, \dots$:

$$\text{index} = (h(\text{page number}) + i) \bmod 17$$

รูปแบบการหยุดการค้น

- เจอช่อง Occupied ที่ page number ตรงกับที่ต้องการ
- เจอช่อง Empty ต้องแทรกใหม่
- เจอช่อง Deleted ให้จำตำแหน่งนี้ไว้เป็นตัวเลือกแรกสำหรับแทรก และให้เดินต่อไปจนกว่าจะเจอ Occupied ที่ page number ตรง หรือ Empty ถ้าสุดท้ายไม่พบ ให้แทรกที่ตำแหน่ง Deleted แรกที่เจอ

Hash Function สำหรับ Page Table:

$$\text{digitHash}(n) = \begin{cases} 0 & \text{ถ้า } n = 0 \\ (n \bmod 10) \cdot 3 + \text{digitHash}(\lfloor \frac{n}{10} \rfloor) & \text{กรณีอื่นๆ} \end{cases}$$

$$h(\text{page number}) = \text{digitHash}(\text{page number}) \bmod 17$$

การทำ Compaction สำหรับการล้างสถานะ Deleted ถ้า probe ครบ 17 รอบ ครอบคลุมทุก index แล้วแล้วยังไม่เจอ Empty หรือ Occupied ที่ page number ตรงแล้วให้ เปลี่ยนทุกช่องที่เป็น Deleted ในตารางทั้งหมดกลับเป็น Empty เริ่ม probe ใหม่ตั้งแต่ $i = 0$ อีกครั้งสำหรับ page number เดิม

Translation Lookaside Buffer (TLB) เป็น Hash table ที่ cache ผลลัพธ์ สำหรับเก็บผลลัพธ์ก่อนหน้าการไปค้นดูใน Page Table เพื่อเพิ่มความเร็วการเข้าถึงหน่วยความจำ โดยประกอบด้วย page number, frame number, valid (บ่งบอกว่า page นี้ยังอยู่ในหน่วยความจำหรือไม่) สำหรับ Hash function และเทคนิคการป้องกันการชนกันของ page ที่นำมา cache ดังนี้

Hash Function สำหรับ TLB:

$$\text{index} = \text{page number} \bmod 4$$

ถ้ามีข้อมูลของ page number อื่นอยู่ในช่องนั้นอยู่แล้ว ให้เขียนทับทันที

Free Frame Stack เป็น Stack ที่เก็บ frame number ที่ยังไม่ถูกใช้งาน โดยกำหนดให้มีขนาด 4 frame ในตอนเริ่มต้นให้ถือว่าทุก frame number (0, 1, 2, 3) อยู่ใน Stack โดยที่ frame 0 อยู่ข้างล่างสุด และ frame 3 อยู่ข้างบนสุดของ Stack

Backing store (non-volatile) เป็นช่องหน่วยความจำขนาดเท่ากับจำนวนทั้งหมดของ page number ใช้เพื่อเก็บค่า ในตอนที่ page ถูกถอดออกจากการใช้งานจะต้องเขียนข้อมูลที่ page นั้นเก็บกลับคืนมาที่ Backing store

ขั้นตอนการถอด Logical address

1. เช็คที่ TLB หากเจอ และ valid ให้อัปเดตค่า access counter เพิ่มขึ้น 1 ใน Page Table ที่ page number นั้นอยู่
2. ถ้าไม่เจอใน TLB ให้ค้นหาใน Page Table ตามรูปแบบการค้นหาที่กำหนด
 - (a) ถ้าเจอให้อัปเดตค่า access counter เพิ่มขึ้น 1
 - (b) หากไม่เจอให้ เกิด Page Fault
 - i. หาก Free Frame Stack ยังมี frame เหลือให้หยิบมาใช้
 - ii. หาก Free Frame Stack ไม่เหลือ frame ต้องเก็บบาง page ที่ใช้อยู่กลับลง Backing Store ก่อนการแทรก page ใหม่
 - iii. แทรกช่องใหม่ ตั้ง dirty = 0, access counter เริ่มที่ 0 แล้วบวก 1 จากการ access ครั้งนี้ และ cached value = Backing Store ในตำแหน่งที่ของ page number นั้น ๆ
 - (c) อัปเดต TLB หลังจากแทรก page ใหม่

ขั้นตอนการเลือก victim page เพื่อเก็บกลับ Backing Store สรรวจทุกช่องที่เป็น Occupied เรียงตามตำแหน่ง index ในตาราง

- หาช่องที่ access counter เป็นเลขคู่และมีค่าน้อยที่สุด เลือกช่องนั้นเป็น victim
- ถ้าไม่มีช่องไหน access counter เป็นเลขคู่เลย เลือกช่องที่ access counter น้อยที่สุด (ถ้าเสมอกัน เลือกช่องที่มี table index น้อยที่สุด)

เมื่อเลือก victim page ที่จะเก็บกลับ Backing Store ได้แล้ว:

- ถ้า victim page มีสถานะ dirty ให้เก็บค่าของ victim page ที่อยู่ใน cached value ลง Backing Store ในตำแหน่งของ victim page number ก่อน
- ตั้งสถานะของ ช่องที่ victim page number นั้นเก็บอยู่ใน page table เป็น Deleted
- ถ้า TLB มีช่องที่เก็บ victim page number อยู่ให้ตั้ง valid เป็น 0 ในช่องนั้น ๆ
- frame ของ victim จะถูกนำไปใช้กับ page ใหม่ทันที (ไม่ผ่าน Free Frame Stack)

โจทย์

ให้เขียนโปรแกรมเพื่อประมวลผลคำสั่ง และติดตามสถานะการทำงานของระบบ

ข้อมูลนำเข้า

บรรทัดที่ 1: จำนวนเต็มบวก n แทนจำนวนคำสั่งที่ต้องประมวลผล โดยที่ $1 \leq n \leq 10^5$

จำนวน n บรรทัด: 1 ใน 4 คำสั่งต่อไปนี้ โดยที่ a หมายถึง Logical address ($0 \leq a \leq 1048576$) และ b หมายถึงค่าที่ต้องการเขียนลงหน่วยความจำ ($-10^9 \leq b \leq 10^9$)

- READ a เข้าถึงหน่วยความจำ
- WRITE a b เขียนค่าลงหน่วยความจำ ตั้ง cached value = b และตั้งว่า page นั้นมีสถานะ dirty
- READVALUE a อ่านค่าข้อมูลในหน่วยความจำ
- REPORT รายงานสถานะของระบบ

ข้อกำหนดการแสดงผล

เมื่อดำเนินการคำสั่ง READ / WRITE / READVALUE จะแสดงผลสถานะอย่างใดอย่างหนึ่งดังต่อไปนี้

```
TLB HIT vpn=<vpn> frame=<frame> offset=<offset>
PAGE TABLE HIT vpn=<vpn> frame=<frame> offset=<offset>
PAGE FAULT vpn=<vpn> frame=<frame> offset=<offset> (no eviction)
PAGE FAULT vpn=<vpn> frame=<frame> offset=<offset> evict vpn=<victim_vpn>
```

หากการถอด address เกิด Page Fault และ victim page นั้นมีสถานะ dirty ให้พิมพ์บรรทัดนี้ก่อนบรรทัด Page Fault เสมอ

```
WRITEBACK vpn=<victim_vpn> value=<victim_cached_value>
```

ถ้าคำสั่งเป็น READVALUE ให้พิมพ์บรรทัดนี้เพิ่มต่อท้ายสุดเสมอ

```
VALUE vpn=<vpn> value=<cached_value>
```

ถ้าคำสั่งเป็น REPORT ให้พิมพ์ตามรูปแบบนี้ โดย Page Table ให้พิมพ์จาก index น้อยไปมาก และพิมพ์เฉพาะช่องที่ Occupied และ TLB ให้พิมพ์ทุกช่องตั้งแต่ index 0 - 4

```

PAGETABLE
<index> <vpn> <frame> <dirty> <access_counter> <cached_value>
...
TLB
<slot> <vpn> <frame> or <slot> EMPTY
...
FREE
<frame top> <frame next> ... or EMPTY

```

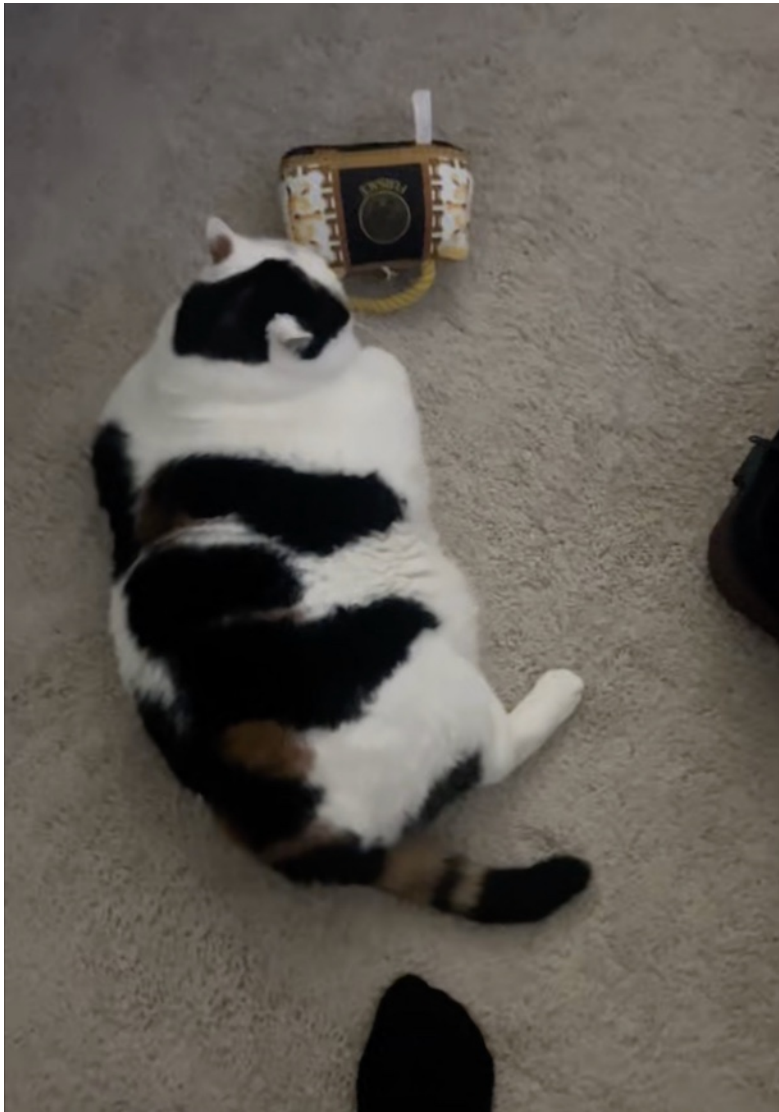
ตัวอย่างข้อมูลนำเข้าและส่งออก

Input	Output
9	PAGE FAULT vpn=8 frame=3 offset=50 (no eviction)
READ 2098	PAGE FAULT vpn=3 frame=2 offset=10 (no eviction)
WRITE 778 100	TLB HIT vpn=8 frame=3 offset=99
READ 2147	PAGE FAULT vpn=25 frame=1 offset=5 (no eviction)
WRITE 6405 555	PAGE FAULT vpn=40 frame=0 offset=200 (no eviction)
READ 10440	PAGE TABLE HIT vpn=8 frame=3 offset=1
WRITE 2049 777	WRITEBACK vpn=25 value=555
READ 25421	PAGE FAULT vpn=99 frame=1 offset=77 evict vpn=25
READVALUE 6405	PAGE FAULT vpn=25 frame=1 offset=5 evict vpn=99
REPORT	VALUE vpn=25 value=555 PAGETABLE 4 25 1 0 1 555 7 8 3 1 3 777 9 3 2 1 1 100 12 40 0 0 1 0 TLB 0 8 3 1 25 1 2 EMPTY 3 EMPTY FREE EMPTY

(จำนวนกรณีทดสอบ: 11 ชุด)

ข้อจำกัดพิเศษ

- สำหรับภาษา C++ อนุญาตให้ใช้แค่ library <iostream> และ <vector> สำหรับการใส่ library อื่น ๆ จะได้รับคะแนนในข้อนี้เป็น 0
- สำหรับภาษา C อนุญาตให้ใช้แค่ library <stdio.h> และ <stdlib.h> สำหรับการใส่ library อื่น ๆ จะได้รับคะแนนในข้อนี้เป็น 0



When I came to you with those calculations